SMU Singapore Management University | School of Computing and Information Systems

Computational Thinking with Python

Jean Chen **James Koh**

{jeanchen, jameskohby}@smu.edu.sg

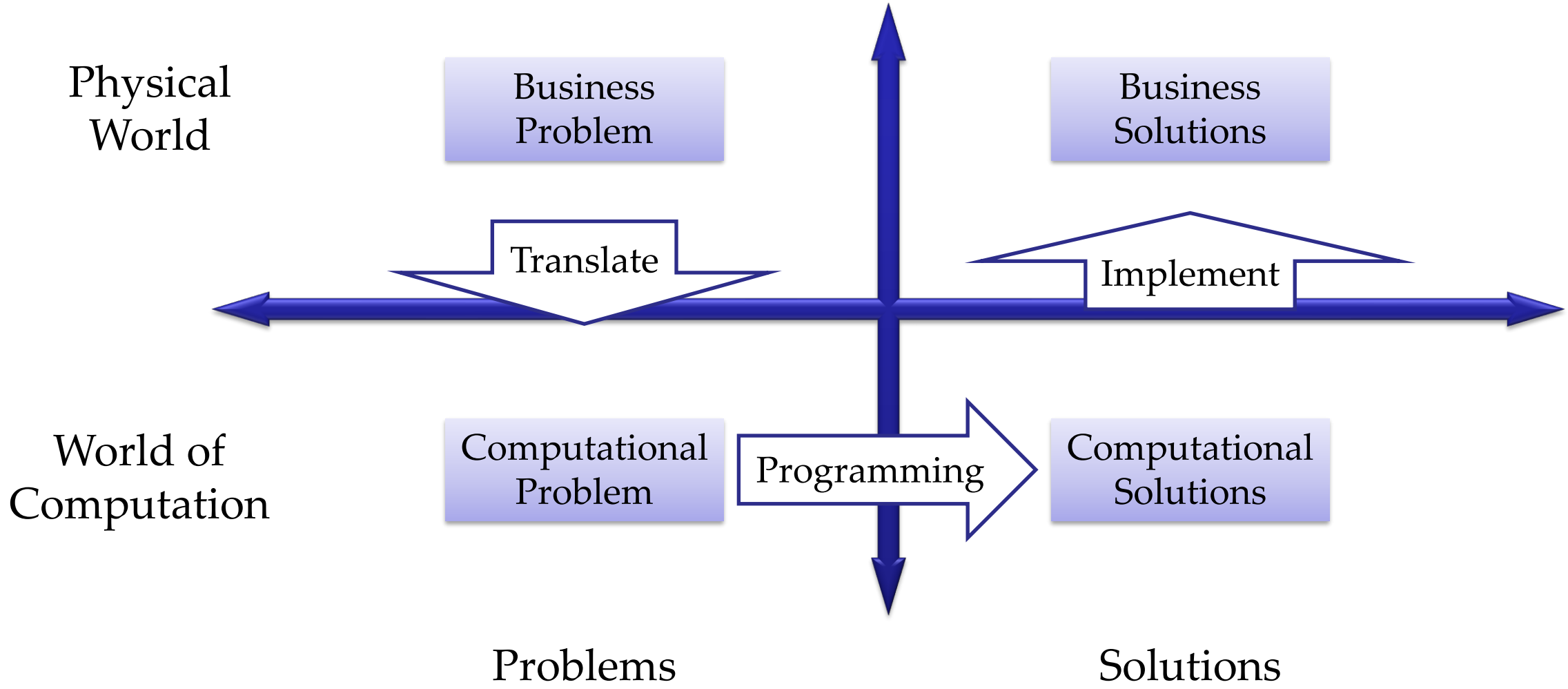
Why Computational Thinking?

- What are the business use cases?
  - Throughout the course, common questions arise regarding the business use cases.

Decomposition	• Project Management: decompose into smaller tasks • Marketing Strategies: decompose into specific segments
Pattern Recognition	• Sales Forecasting: recognize patterns in sales data • Financial Analysis: recognize patterns in financial statements
Abstraction	• Process Mapping: high-level visual representation • Risk Management: abstract risk factors into simple models
Algorithmic Thinking	• Decision Making: suggest optimal choices with constraints • Operation Management: optimize processes for efficiency

© MITB, SMU

Why Programming? – Problem Solving



© MITB, SMU

Computational Thinking

- Decomposition
- Pattern Recognition
- Abstraction
- Algorithmic Thinking

Monte-Carlo Sim

This class

- Simulation to support decision-making

© MITB, SMU

Money without working hard?
A TOTO Jackpot Simulation

How is TOTO played?

- You pay

bet	pay
Choose 6 numbers from 1, 2, …, 49	$1
Choose 7 numbers	$7
Choose 8 numbers	$28
Choose 9 numbers	$84
Choose 10 numbers	$210
…	…

- You pay

6 winning numbers	1 additional number

- You may win

	condition	prize
Group 1	Match all 6 winning numbers	min(1 million, 38% of prize pool divided by num of people in group 1)
Group 2	Match 5 winning numbers plus the additional number	8% of the prize pool divided by num of people in group 2
Group 3	Match 5 winning numbers	5.5% of the prize pool divided by num of people in group 3
…	…	…

Group	main	Add
1	6	
2	5	+ 1
3	5	
4	4	+ 1
5	4	
6	3	+ 1
7	3	

#13 - 2**6 - 0 = 1
#13 - 2**5 - 1 = 2
#13 - 2**5 - 0 = 3
#13 - 2**4 - 1 = 4
#13 - 2**4 - 0 = 5
#13 - 2**3 - 1 = 6
#13 - 2**3 - 0 = 7

© MITB, SMU

Sample vs Choices

→ will not have repeating numbers.

- random.sample() picks unique elements (no duplicates)
  - Sampling without replacement

→ may have numbers repeated.

- random.choices() may pick the same item multiple times
  - Sampling with replacement

- Which one should you use if you want to pick 6 numbers from 1 to 49?

- Which one should the system use if 6 winning numbers and 1 additional number are to be generated?

© MITB, SMU

Exercise 9-1: Simple Simulator

Repeat

1. Pick 6 numbers
2. System generates 6 winning numbers + 1 additional number
3. Compare
   - How likely I won something?
   - Probabilistic distribution of prizes?

```
def calculate_prize(num_list, winning_list):
    match_count, match_additional = 0, False
    for i in num_list:
        if winning_list[i] == 1:
            match_count += 1
        elif winning_list[i] == 2:
            match_additional = True

    if match_count == 6:
        return 1
    elif match_count >= 3:
        return 13 -2*match_count - match_additional
    else:
        return 0
```


© MITB, SMU

Area Estimation with Computational Thinking

Exercise 9-2: Area Calculation

- Easy?

- Many formulas for calculating area:
  - Triangle: base-height formula, Heron's formula
  - Rectangle, square, parallelogram
  - Trapezium / Trapezoid
  - Circle, sector, ellipse

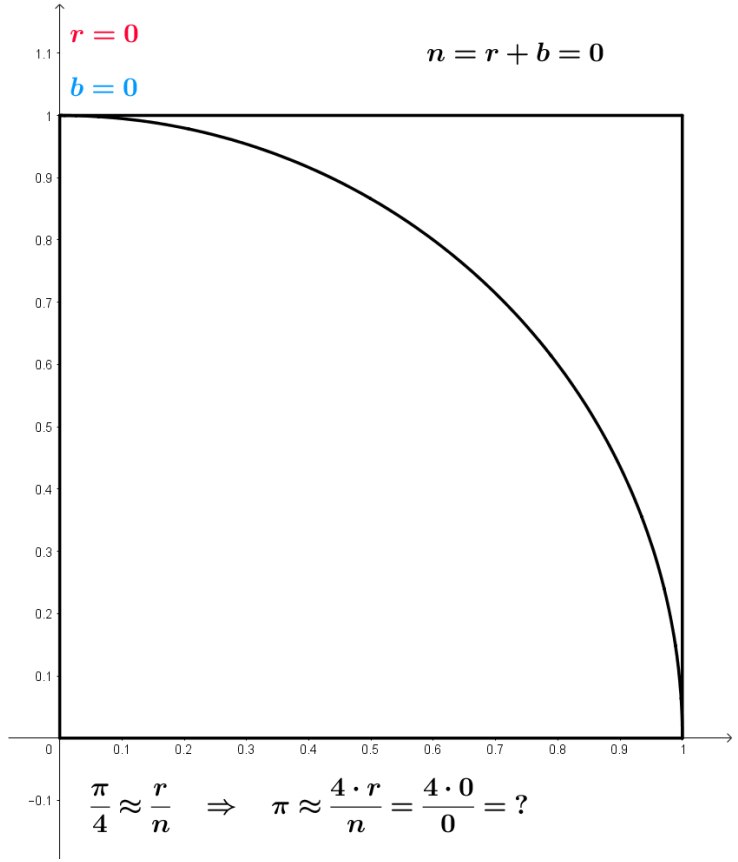
- Difficult to apply in practice, why?
  - How to calculate the area of an irregular shape, e.g., a land plot?

© MITB, SMU

Monte Carlo Method

Monte Carlo Method:

1. Define a domain of possible inputs
   - A set of 2D points for 2D area estimation
2. Generate inputs randomly from a probability distribution over the domain
   - Uniform distribution in a rectangular area
3. Perform a deterministic computation of the outputs
   - Inside or outside the shape?
4. Aggregate the results
   - Count the number of points falling inside of the shape
   - Estimate the area according to the probability

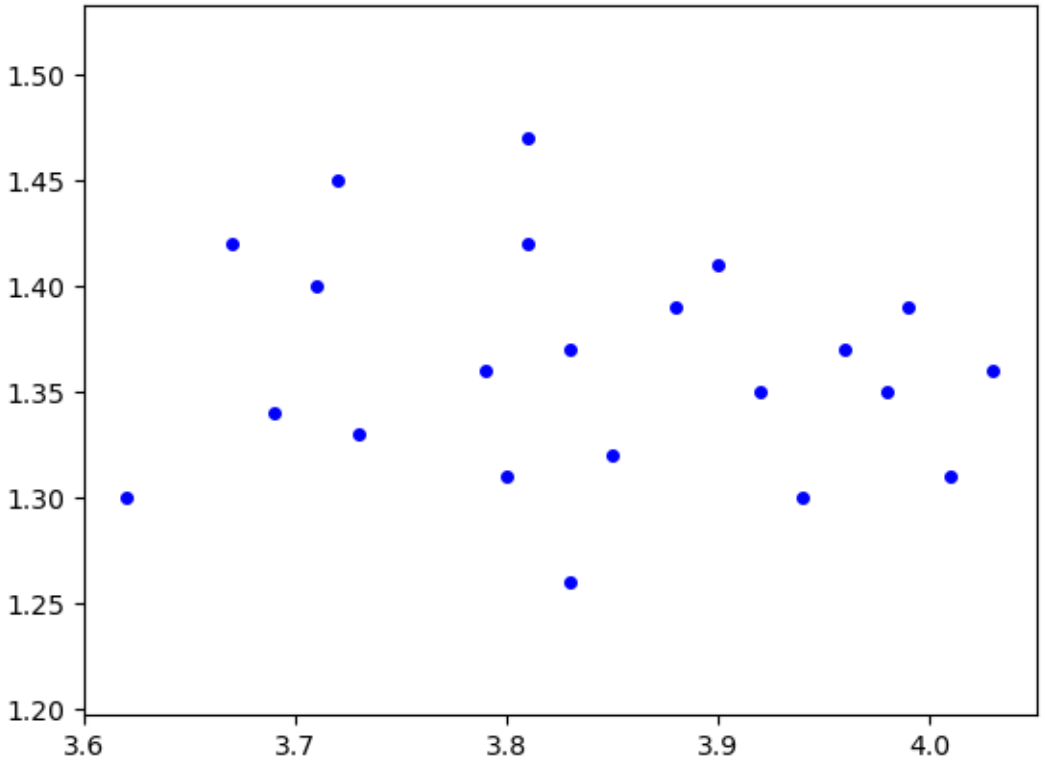


en.wikipedia.org/wiki/Monte_Carlo_method

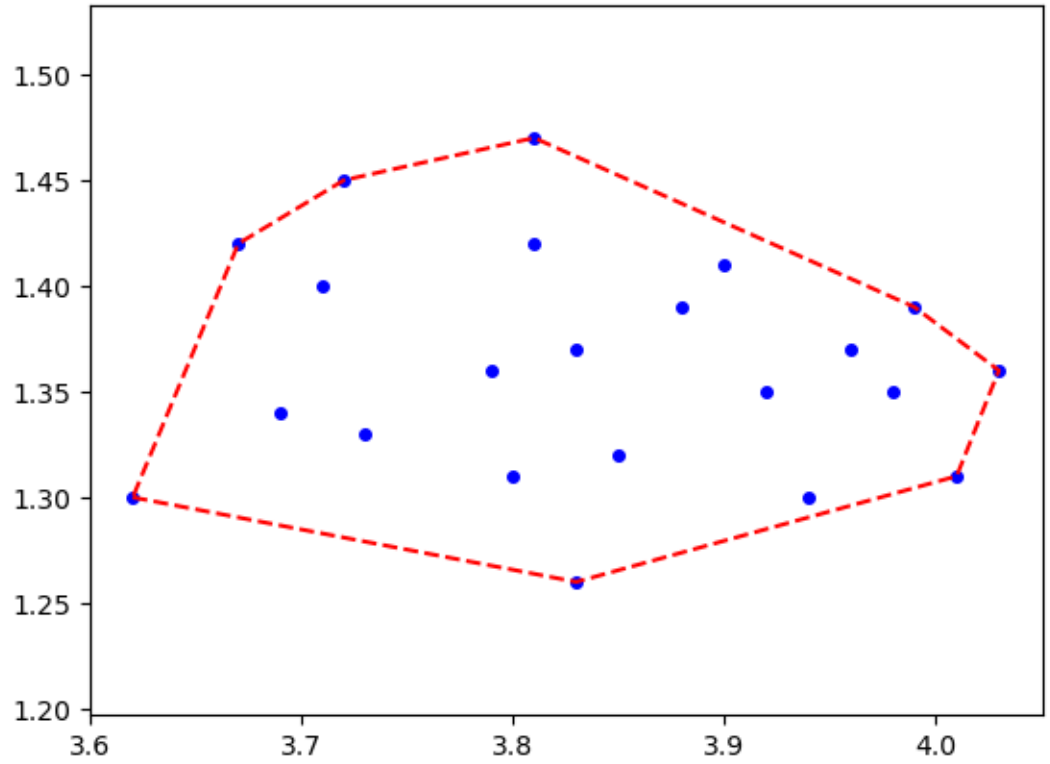
© MITB, SMU

Convex Hull

- An irregular shape is defined by a sequence of 2D points on its boundary
- Internal points do not matter



- Convex: a line segment connecting any two internal points are entirely inside



© MITB, SMU

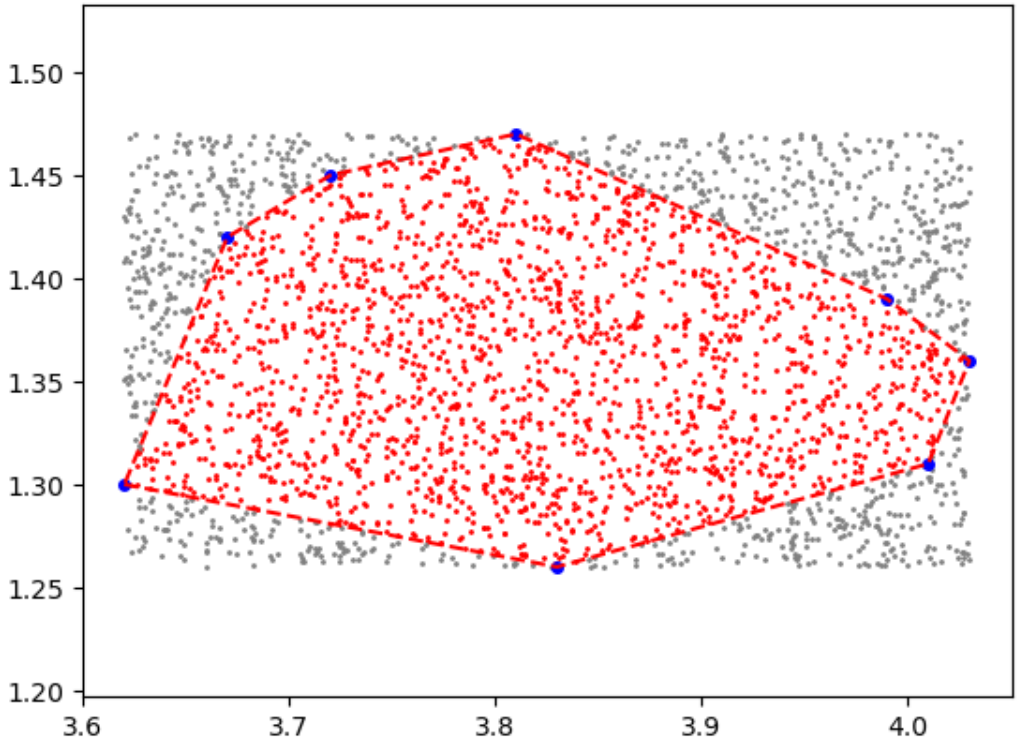
Inside or outside?

- Finding convex hull is easy (by using `scipy`):

```
import numpy as np
from scipy.spatial import ConvexHull
vertices = np.array(a_list_of_coordinates)
hull = ConvexHull(vertices)
num_hull_vert = len(hull.vertices)
```

comes with attributes — can use to find vertices.

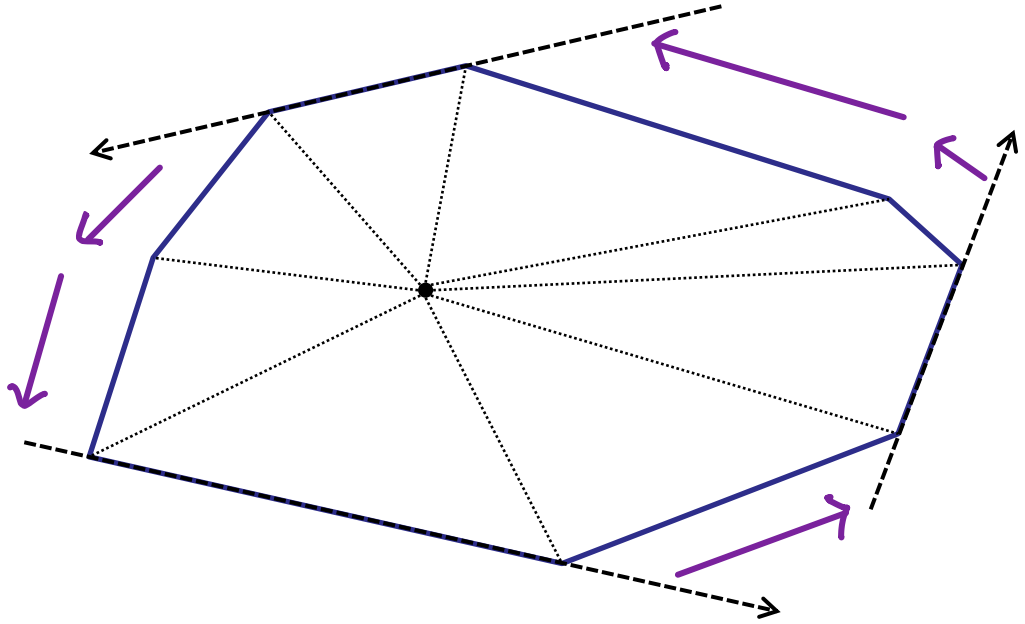
- How to tell a point falls inside or outside?

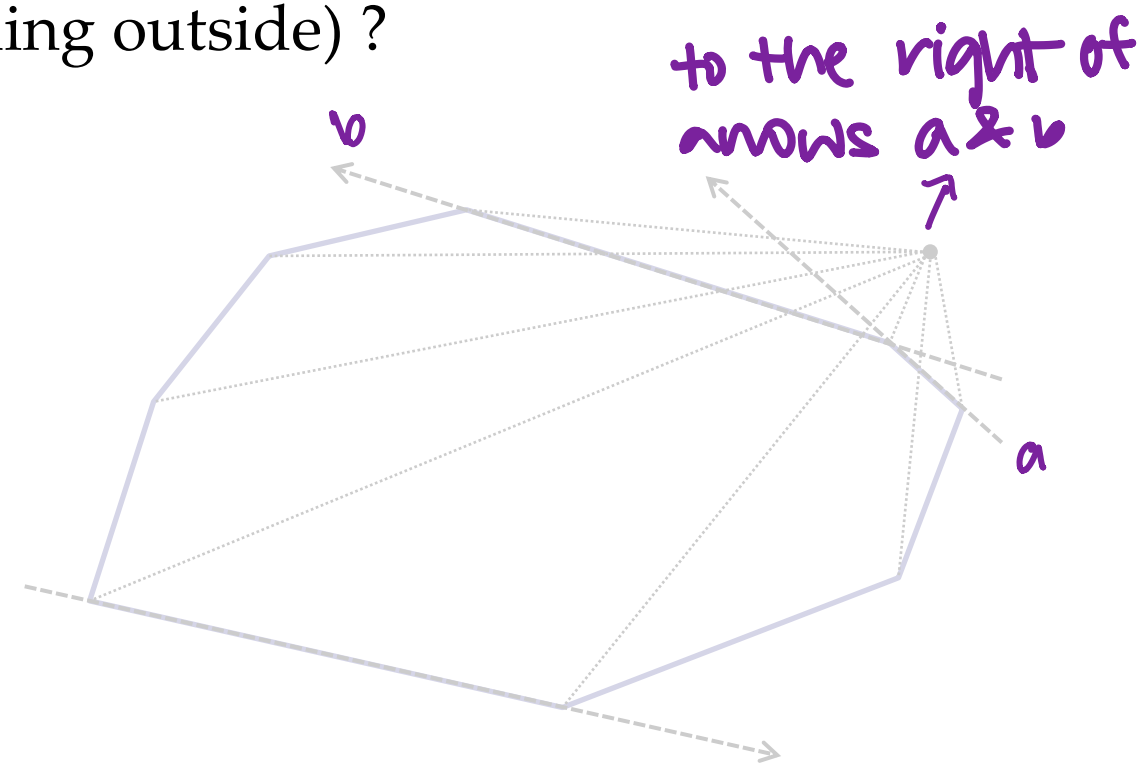


© MITB, SMU

Inside or outside?

- An internal point (a point falling inside) will appear on the left-hand side of each segment of the convex hull when the vertices of the hull are described in an anticlockwise order.
- What about an external point (a point falling outside) ?

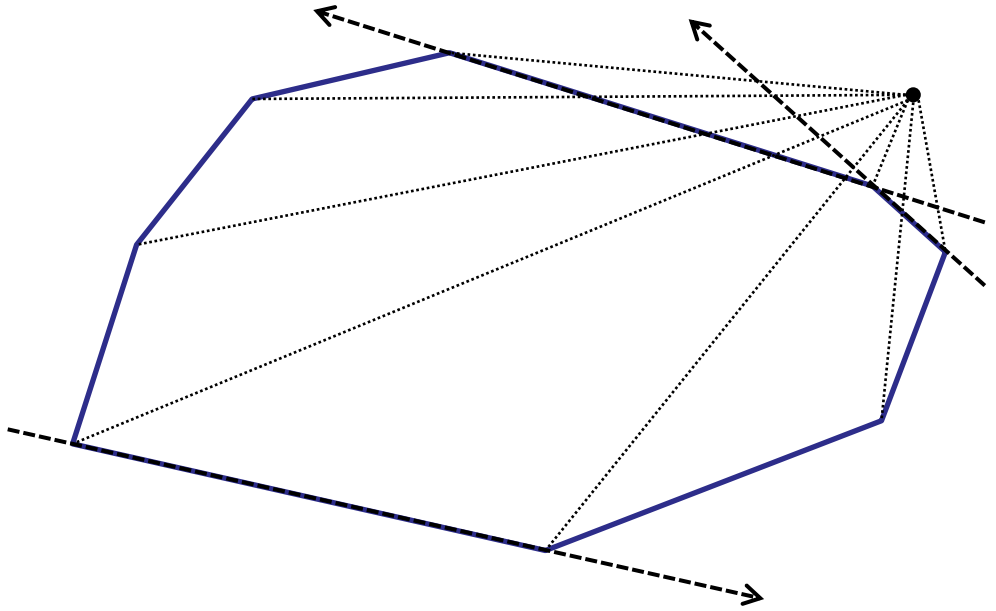




© MITB, SMU

Inside or outside?

- An external point is considered outside if it appears on the right-hand side of a segment of the convex hull, even if only once.
- Which logical operator shall we use for internal and external points?

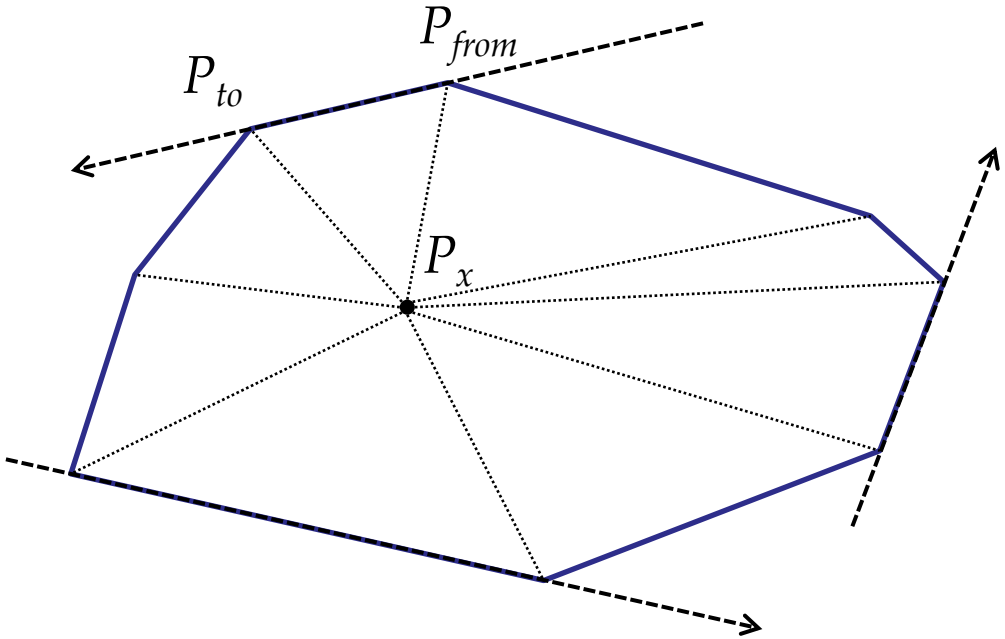


© MITB, SMU

Relationships of a point and a line segment

- The left-hand side or the right-hand side can be determined mathematically by a linear algebra formula

$$\det\left(\begin{bmatrix} P_x.x & P_x.y & 1 \\ P_{fr}.x & P_{fr}.y & 1 \\ P_{to}.x & P_{to}.y & 1 \end{bmatrix}\right) \begin{cases} > 0 \rightarrow \begin{cases} \text{anticlockwise} \\ P_x \text{ on the left-hand side} \end{cases} \\ = 0 \rightarrow \quad P_x \text{ on the line from } P_{fr} \text{ to } P_{to} \\ < 0 \rightarrow \begin{cases} \text{clockwise} \\ P_x \text{ on the right-hand side} \end{cases} \end{cases}$$

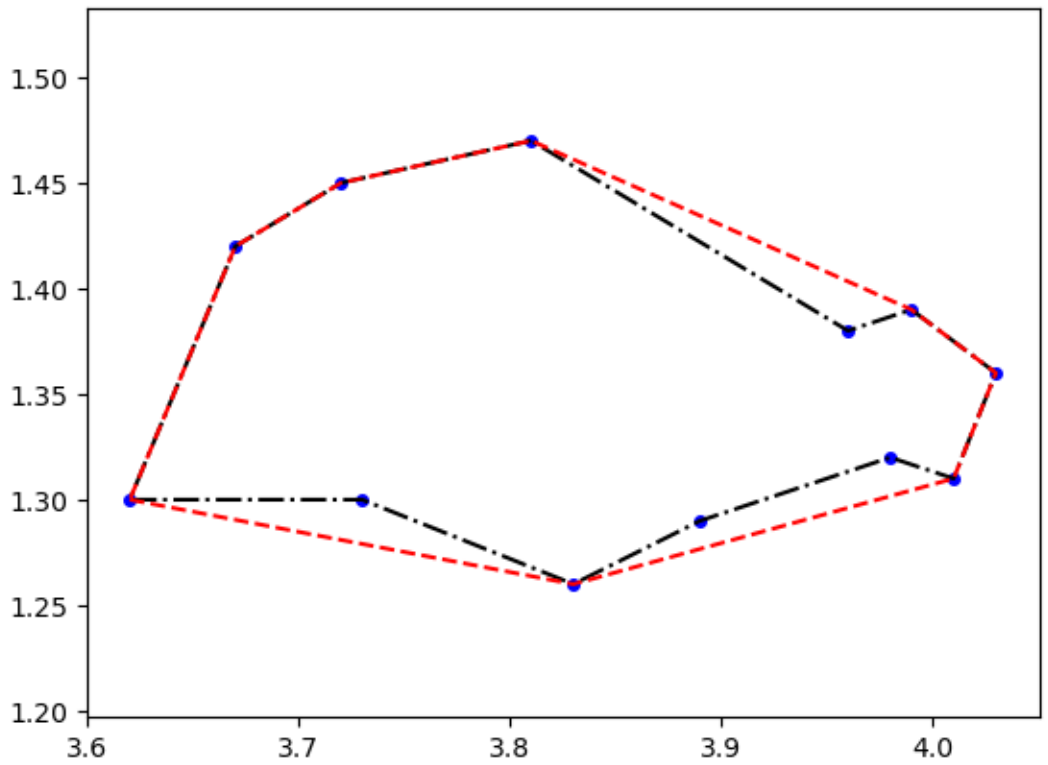


Optional: It is just an application of formula, functions are given.

© MITB, SMU

Not all shapes are convex

- Convex: a line segment connecting any two internal points are entirely inside.
- Concave polygons: polygons which are non-convex.



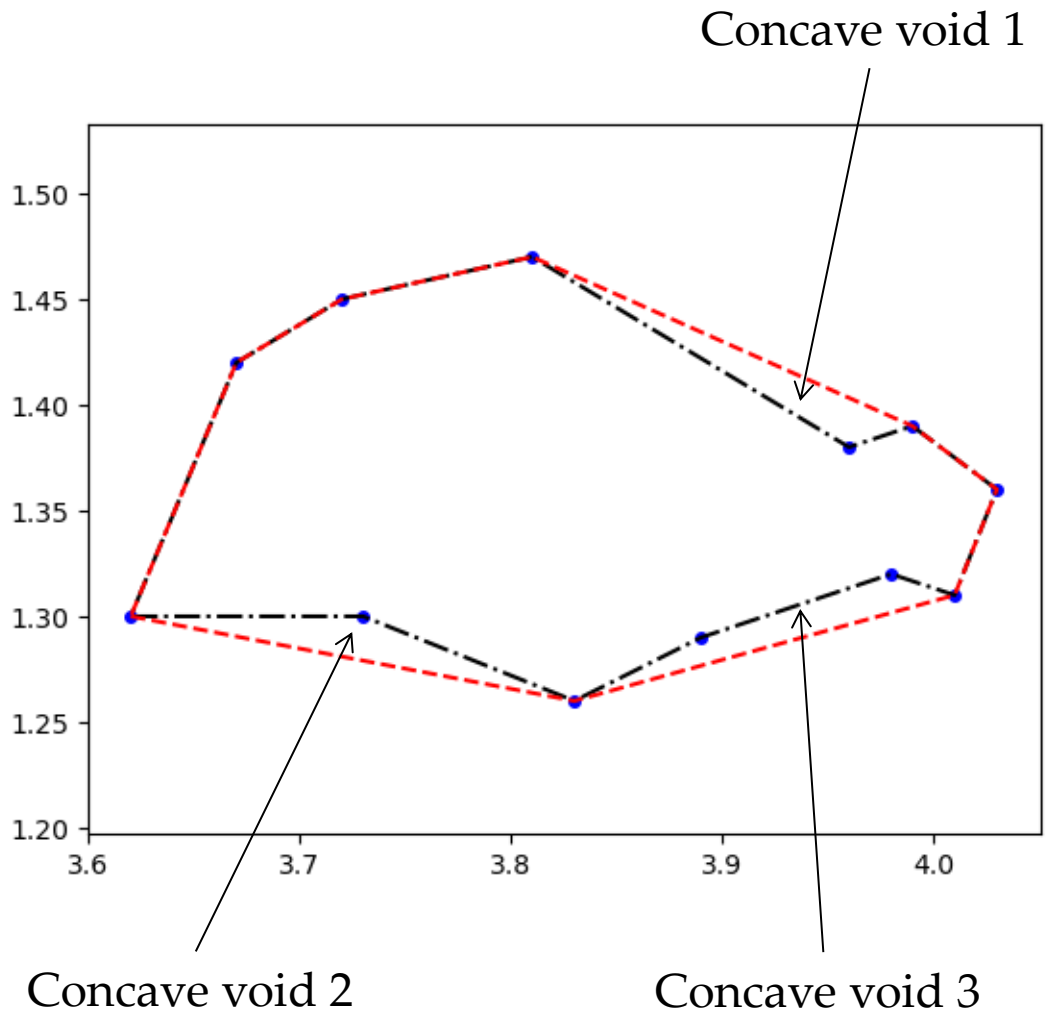
boundary = np.append(hull.vertices, hull.vertices[0])
the vertices
add back the 1st one to close the loop
[0,0] [0,1] [1,1] [1,0]
1 2 3

© MITB, SMU

Concave Polygons

- From the area of the convex hull
  - Subtract the area of concave void 1
  - Subtract the area of concave void 2
  - Subtract the area of concave void 3

- However, using Monte Carlo is easier, just need to amend the standard for internal points.



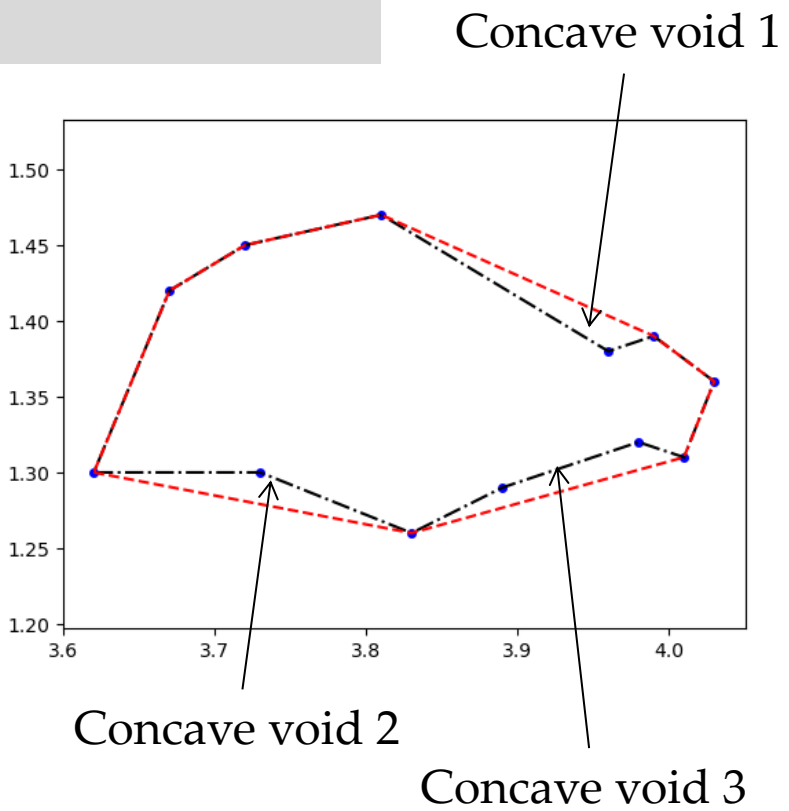
© MITB, SMU

Concave Polygons


```
for _ in range(total_sample):
    x_rand, y_rand =   ## uniform within in the rectangle box
    if inside((x_rand, y_rand), convex_hull) \
       and not any([inside((x_rand, y_rand), i) for i in concaves]):
        count_inside += 1
```

- `is_inside = True` if it is inside the convex hull, and not inside any of the concave voids

- Now, how to find concave voids?



© MITB, SMU

What if a concave void itself is concave?

- Abstraction:
  - A concave void can be any polygon we started with
  - We can check if a point is inside a concave void, just as we check if it is within the initial polygon.

```
def inside_convex_hull(self, px):
    return all([left_of_line(px, self.vertices[self.boundary[i]], \
                                 self.vertices[self.boundary[i+1]]) \
                for i in range(self.num_hull_vert)])

def inside_polygon(self, px):
    return inside_convex_hull(self, px) \
           and not any([inside_polygon(i, px) for i in self.concaves])
```


© MITB, SMU

Wooclap!

What is the output of the below print statement?

```
def func(a, b):
    while b:
        temp = b
        b = a % b
        a = temp
    return a

print(func(12, 18))
```

a. 0
b. 2
c. 6
d. 12

Wooclap! Explanation

- The previous question is a way to compute the Greatest Common Divisor.
- For more details, you can refer to Euclidean Algorithm

```
def func(a, b):
    while b:
        temp = b
        b = a % b
        a = temp
    return a

print(func(12, 18))
```

Portfolio Management with Computational Thinking

Exercise 9-3: Portfolio Management

- Portfolio management is to manage a collection of investments to meet specific financial goals.
- It involves selecting and overseeing various asset classes, such as stocks, bonds, real estate, and cash, to achieve a desired risk-return profile:
  - Asset Allocation: Determining how to distribute investments across different asset classes based on risk tolerance and investment objectives.
  - Investment Strategy: Deciding on a strategy, such as passive (index investment) or active management (selecting individual stocks).
  - Risk Management: Identifying and mitigating potential risks associated with investments.
- Portfolio management aims to optimize returns while controlling risk according to the investor's objectives.

© MITB, SMU

Passive Asset Classes

Typically involve a buy-and-hold strategy, minimizing frequent trading and aiming to replicate the performance of a market index rather than outperform it.

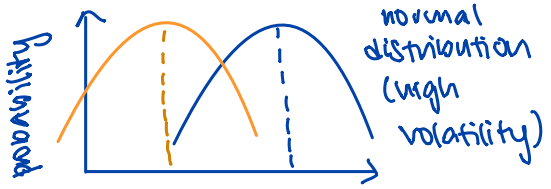
- Stock index
  - Higher potential returns with significant market volatility.
- Bond index
  - Lower returns with more stability, but still carries interest rate and credit risks.
- Real Estate Investment Trusts (REITs)
  - High dividend yields with volatility influenced by real estimate market condition.
- Commodity futures
  - High profit potential and high volatility, with significant leverage risks.

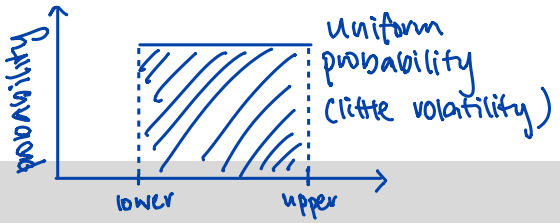
© MITB, SMU

Stock Index + Bond Index

- In this example, we consider the most common scenario: a portfolio made up of a stock index and a bond index.
- The 5-year return will be simulated instead of the 1-year return, allowing us to understand the impact of economic fluctuations.

- Suppose two models for the stock index and the bond index are given in percentages, as below:





Gaussian / Normal Distribution

std dev

```
growth = [random.uniform(1, 4) for _ in range(5)]
si = [random.gauss(2.2 * math.exp(0.4*x) - 2.0, 0.4 * math.exp(0.4*x)) for x in growth]
bi = [random.uniform(x + 0.2 * math.exp(0.2*x), x + 0.8 * math.exp(0.2*x)) for x in growth]
```

↳ uniform distribution.

© MITB, SMU

Example returns

- Example returns after adjusting for inflation

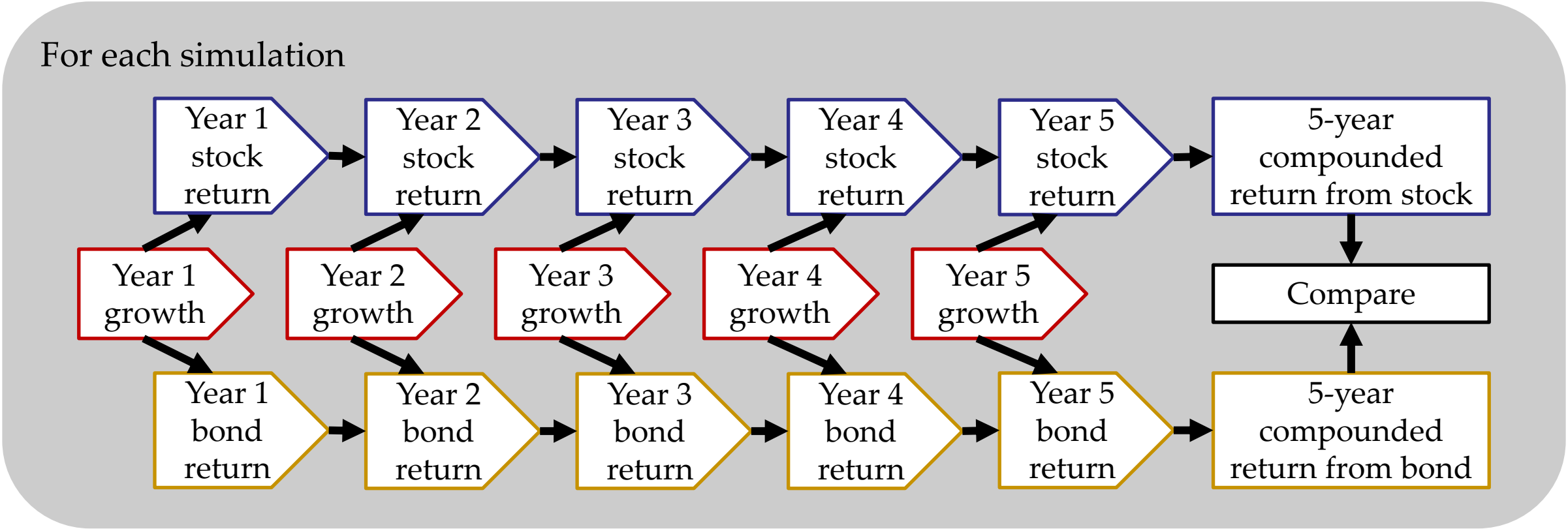
	Growth = 1%	Growth = 2%	Growth = 3%	Growth = 4%
Return from the stock index	~N(1.28%, 0.60%)	~N(2.90%, 0.89%)	~N(5.30%, 1.33%)	~N(8.90%, 1.98%)
Return from the bond index	~U(1.24%, 1.98%)	~U(2.30%, 3.19%)	~U(3.36%, 4.46%)	~U(4.45%, 5.78%)

```
growth = [random.uniform(1, 4) for _ in range(5)]
si = [random.gauss(2.2 * math.exp(0.4*x) - 2.0, 0.4 * math.exp(0.4*x)) for x in growth]
bi = [random.uniform(x + 0.2 * math.exp(0.2*x), x + 0.8 * math.exp(0.2*x)) for x in growth]
```


© MITB, SMU

Simulation Strategy

- A single growth variable determines the ranges for generating returns from the stock index and the bond index each year.



© MITB, SMU

Simulation Strategy

- A single growth variable determines the ranges for generating returns from the stock index and the bond index each year.

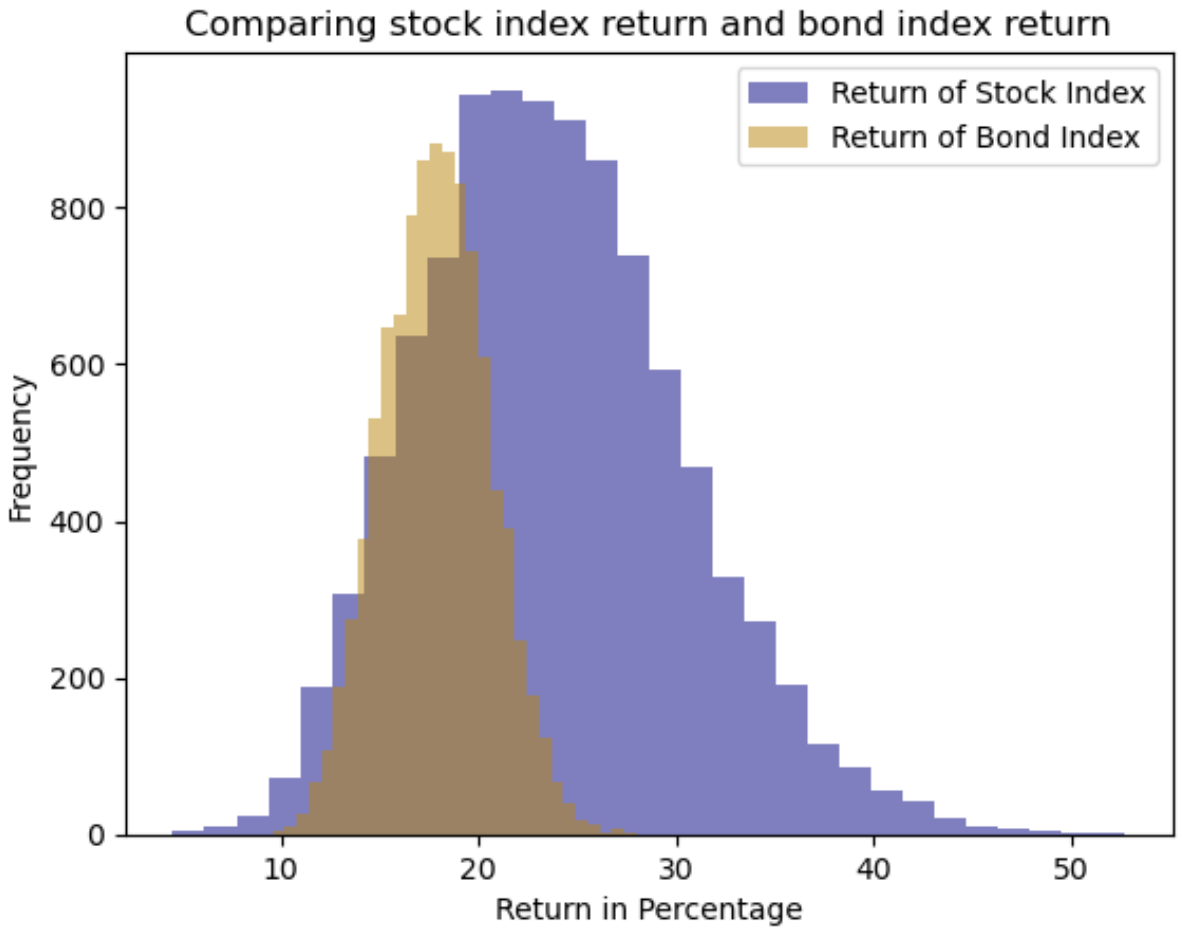
```
def generate_return(num_year):
    growth = [random.uniform(1, 4) for _ in range(num_year)]
    si = [random.gauss(2.2 * math.exp(0.4*x) - 2.0, 0.4 * math.exp(0.4*x)) for x in growth]
    bi = [random.uniform(x + 0.2 * math.exp(0.2*x), x + 0.8 * math.exp(0.2*x)) for x in growth]
    sr, br = 1.0, 1.0
    for i in range(num_year):
        sr *= (1.0 + si[i] / 100)
        br *= (1.0 + bi[i] / 100)
    return sr - 1, br - 1
```

cumulative

© MITB, SMU

Comparing by histograms

- The stock index performs better, but subject to high volatility
- The bond index performs worse, but more stable
  - The worst scenarios are not as bad as the worst scenarios of the stock index
- A portfolio made up of the stock index and the bond index should outperform the bond index, but gives a lower volatility



© MITB, SMU

Target return

- What is the x, such that the probability of having a return greater than or equal to x, is 70%?

$$\Pr[\text{return} \geq x] = 70\%$$

- What is the y, such that the probability of having a return greater than or equal to y, is 80%?

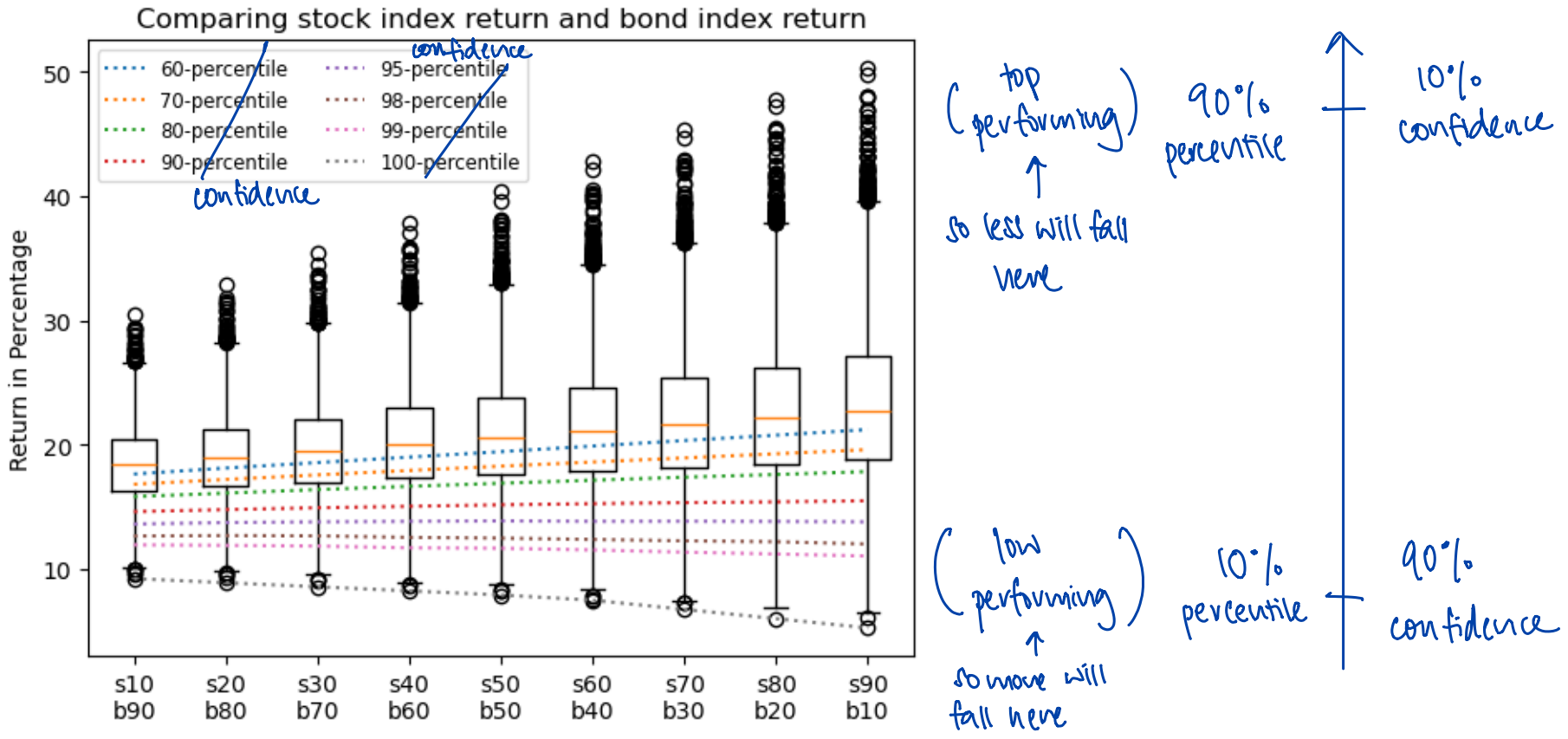
$$\Pr[\text{return} \geq y] = 80\%$$

- This requires a percentile query using `np.percentile`.

© MITB, SMU

Target return from a portfolio

- A portfolio is made up of a stock index and a bond index.
- How does the target return changes with the proportion of the stock index?



© MITB, SMU

Wooclap!

What is the output of the below print statement?

Hint: If you think this is valid, try testing the func with n = 1, 2, 3 and see if you can spot a pattern!

```
def func(n):
    row = [1]
    [row.append(row[i] * (n-i) // (i+1)) for i in range(n)]
    return row

print(func(4))
```

Wooclap!

Given the below patterns, which pattern, when filled into the blank below, is the SHORTEST pattern (using len) such that the result of the match is ['is', 'The', 'Yes']?

Hint: The keyword is SHORTEST, meaning that there could be more than one answer below that gives you the correct match! Take note of what is required in the output too!

```
pattern = r"_____________"
p = re.compile(pattern, re.I)


print(p.findall("THIS is is isn't a test.
The the mistake was obvious. Yes yes!"))
```

a. (\w+) \1\b

b. \b(\w+) \1

c. \b(\w+) \1\b

d. (\w+) \1

Performance Bonus with Computational Thinking

Performance Bonus and Annual Increment

- Performance Bonus
  - Receive at the end of each year
  - Considered as a “short-term reward”

- Annual Increment
  - Receive as an increment of monthly salary in the subsequent year
  - Impact all subsequent years
  - Considered as a “long-term reward”

- Which one do you choose?

© MITB, SMU

Determining factors

- Diligence: personal effort
- Economy: the overall economic condition of that year



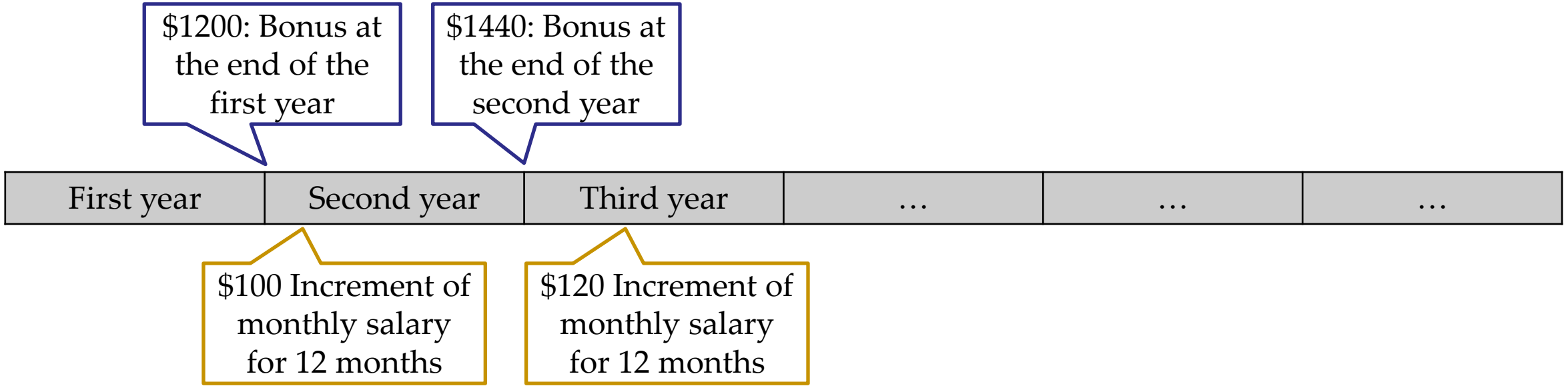
© MITB, SMU

Exercise 9-4a: P.B. versus A.I.

Under the same performance indicators from one individual

- How much performance bonus will he get over a period of 5 years?

- How much more will his annual increment bring to him for the next 5 years (exclusive of the first year) ?



© MITB, SMU

Generating Employee's Bonus


```
salary = [random.randint(6000000, 8000000) / 100 for _ in range(num_employ)]

diligence = random.sample(possible_diligence, num_employ)
performance = [max(0.0, random.gauss(0.1 * p, 0.01 * p)) for p in diligence]

print('\n'.join(['expected year-end bonus: %.2f' % (salary[i] * performance[i]) \
                                                  for i in range(num_employ)]))

year_goodness = random.uniform(0.8, 1.2)
print('adjusted by the year goodness:')
print('\n'.join(['    adjusted year-end bonus: %.2f' \
       % (salary[i] * performance[i] * year_goodness) for i in range(num_employ)]))
```


© MITB, SMU

Exercise 9-4b: What if one resigns?

- What is the rationale behind the CFO allowing employees to save their year-end bonuses for annual increments?

- If someone resigns, the increments for the remaining months are forfeited.

© MITB, SMU

Exercise 9-4c: a mixture of P.B. and A.I.?

- What if an employee is allowed to opt the ratio of P.B. against A.I.?
  - One can redeem all his year-end bonus every year
  - One can save all bonus for annual increment
  - One can allocate 50% of his bonus to annual increment, and redeem the rest
  - One can also take only 20% of his bonus, make the rest into the annual increment
  - What is a better choice?

© MITB, SMU

Exercise 9-4d: Choices on working attitude

- All simulations are not totally reflective of real-life, any conclusions should be replicated at your own risk

- Consistent mode: work hard every year at 70% of maximum effort, for five years

- Inconsistent mode: work hard for two year consecutively at 100% of maximum effort, then laze (50%) for 3 years

© MITB, SMU

Computational Thinking for Real-life Problems

- Abstraction
  - Understand the flow
  - Model whatever you can
  - Treat others as random variables

- Decomposition
- Pattern Recognition
- Algorithmic Thinking

© MITB, SMU

What's more

- Operations Analytics & Applications
- Process Analytics & Applications
- Algorithm Design & Implementations
- AI Planning & Decision Making
- Multi-agent Systems

© MITB, SMU

SMU
SINGAPORE MANAGEMENT UNIVERSITY

School of
Computing and
Information Systems



– The End –

Thank You